

0. Cómo desarrollar la intuición antes de programar

Es más un método que se entrena que una chispa de inspiración. Algunas cosas concretas que podés hacer antes de escribir código:

1. Cuando el enunciado te da una precondition "gratis", preguntate qué te compra.

Si dice "ordenadas y sin repetidos", esa frase no está de adorno — casi seguro habilita un algoritmo más simple que si no la tuvieras. El reflejo tiene que ser: "¿qué chequeo puedo dejar de hacer porque esto ya me lo garantizan?". Por ejemplo: sin la precondition, sí necesitarías `elem` (buscar si ya lo pusiste). Con ella, comparar las dos cabezas alcanza. Cuando una precondition te resulte "obvia y sin uso", desconfiá — probablemente la estás dejando pasar.

2. Tomate en serio las restricciones de complejidad como restricciones de diseño, no como detalles de examen.

"Un único recorrido" no es una nota al margen: es información que elimina opciones de tu espacio de soluciones antes de pensar en la lógica. Si tu primera idea usa `elem`, `++` en cada paso, o ordenás y después limpiás duplicados, ya sabés que no puede ser la solución pedida — sin necesidad de terminarla para descubrir que está mal. Usalo como filtro temprano.

3. Construite un catálogo mental de patrones recursivos y buscá el "match".

"Combinar dos listas ordenadas en una sola ordenada" es literalmente el paso merge de mergesort. Si ese patrón ya está en tu cabeza, no lo inventás cada vez — lo reconocés. Vale la pena, cuando resolvés un ejercicio así, quedarte con el patrón pelado (no el código específico): "dos punteros, comparo cabezas, avanzo la/las que corresponda". La próxima vez que veas "dos listas ordenadas" el patrón salta solo.

4. Simulá a mano con un ejemplo chico antes de escribir Haskell.

Agarrá `[1,2,3]` y `[2,3,4]` en papel y mezclalos vos, a mano, como si fueran dos mazos de cartas ordenados. Vas a notar que naturalmente mirás solo el tope de cada mazo, nunca "todo lo que ya saqué" — eso es el algoritmo. Traducir un proceso que ya hacés intuitivamente es mucho más fácil que razonarlo en abstracto de una.

5. Antes de tipear la recursión, escribí el invariante en una frase.

Algo tipo: "en cada llamada, prometo que ____". Si no podés completar esa frase, todavía no entendiste bien la estructura recursiva. Por ejemplo: "todo lo que ya emití es menor a las dos cabezas actuales". Este hábito además sirve doble en esta materia: el ejercicio de demostración pide probar una propiedad por inducción — el invariante que usás para diseñar la función es casi literalmente el que después usás para probar cosas sobre ella.

En resumen: la próxima vez que una precondition te resulte "sospechosamente generosa" y encima te limiten la complejidad, tratalo como una pista fuerte de que existe una versión sin búsqueda — y buscá el patrón de "dos punteros" antes de agregar estructuras auxiliares.

1. Recordatorios generales

Antes de escribir una sola línea

- ☐ **Leé cada precondition como una pista de diseño, no como decoración.** Si el enunciado te garantiza algo (orden, unicidad, totalidad de una función, etc.), preguntate qué chequeo te ahorra tener esa garantía. Una precondition que "no usás para nada" es sospechosa — probablemente la estás dejando pasar.
- ☐ **No confíes ciegamente en los datos de ejemplo del enunciado.** Pueden venir mal transcritos (de un PDF escaneado, de un apunte a mano, etc.). Antes de programar contra un ejemplo, auditalo con algo que no dependa de tu propia implementación:
 - ☐ Si el enunciado dice que algo "se repite" con cierto período, dos posiciones congruentes módulo ese período **tienen** que dar el mismo resultado. Si no da lo mismo, el dato está mal.
 - ☐ Si hay un orden impuesto por la propia estructura (una secuencia, una cadena), un elemento no puede aparecer antes que el que lo precede.
 - ☐ Contá cantidades (duraciones, tamaños, pasos) a mano con sumas simples antes de asumir cualquier otra cosa sobre un ejemplo complejo.
- ☐ **Simulá un caso chico a mano en papel** antes de tipear Haskell. Si el algoritmo "se siente" natural haciéndolo vos mismo con un ejemplo concreto, traducirlo a código es fácil; inventarlo en abstracto de una no.

Checklist final antes de dar por terminado un ejercicio

- ☐ **Chequeo "en papel" de sintaxis y tipos** (no hay ghci en el parcial, así que esto se hace a ojo y a mano):
 - ☐ Contá los argumentos de cada aplicación de constructor contra su aridad en el `data` — dedo por dedo si hace falta. Un constructor de aridad 2 usado con un solo argumento es el error más común y el más fácil de pasar por alto.
 - ☐ Contá paréntesis abiertos vs. cerrados en cada línea con llamadas anidadas (`f (g x) (h y)`), en particular alrededor de una llamada recursiva que le pasás como argumento a otra función.
 - ☐ Para cada `let`, verificá que tiene su `in` — no se pueden encadenar dos `let` sin cerrar el primero.
 - ☐ Para cada llamada (recursiva o no) que escribas, mirá la firma `::` de esa función y chequeá que el orden de los argumentos que le pasás coincide de izquierda a derecha.
 - ☐ Recordá que la negación booleana en Haskell es `not`, no `!`.
- ☐ **¿Los tipos cierran?** ¿Le estás dando a cada constructor todos sus argumentos, en el orden correcto? ¿El orden de los argumentos de cada función coincide con su firma?
- ☐ **¿Los nombres de variable representan lo que decís que representan?** Usar el nombre de un parámetro de otra definición en un paso de demo, en vez del nombre real de la variable que tenés fija en ese contexto, es inconsistente aunque el resultado final no cambie.
- ☐ **¿Usaste la HI?** ¿En qué paso exactamente? Si no la usaste en ningún caso inductivo, revisá ese caso.
- ☐ **¿Los casos cubren todos los constructores del tipo?** Contalos contra la declaración de `data`.
- ☐ **¿Verificaste tu implementación contra los ejemplos del enunciado**, no solo contra tu intuición?
- ☐ **¿Hay alguna búsqueda o recorrido extra** (`elem`, recorrer una lista ya generada, etc.) donde el enunciado pedía "único recorrido" o una complejidad acotada?
- ☐ **¿Hay recursión explícita** en una función donde el enunciado pedía usar solo esquemas?

2. "Sin recursión explícita" / esquemas (fold, rec)

- ☐ La restricción es **sobre el tipo que estás recorriendo en la función que estás escribiendo ahora mismo**, no sobre "nada en el programa puede usar recursión". Podés seguir llamando a funciones ya definidas (aunque internamente usen recursión explícita) — eso es aplicar una función, no recurrir vos.
- ☐ Si la función que tenés que escribir recorre un tipo para el que el enunciado **no** te pidió un esquema explícito en otro ejercicio, armate el esquema de ese tipo igual — es un prerequisite implícito, no algo opcional.
- ☐ **fold vs rec**: un `fold` solo te da, en cada campo recursivo, el resultado ya procesado (`b`). Un `rec` te da además la subestructura original sin procesar. Necesitás `rec` en cuanto tengas que volver a mirar la forma de un hijo (no solo su resumen) — típicamente en funciones que "simplifican" o "reconstruyen" comparando contra el original.
- ☐ Pista rápida: "recorro todo y devuelvo un resumen/acumulado" → `fold`. "En algún punto necesito el árbol/lista original, no solo su resultado" → `rec`.
- ☐ Ojo con el **caso base compartido**: un esquema usa un único valor para representar el caso base, aunque ese mismo caso base pueda aparecer en dos lugares distintos de la estructura con un significado distinto en cada uno (por ejemplo, un tipo con dos campos recursivos donde el mismo constructor "vacío" significa cosas distintas según en qué campo aparece). Si eso pasa, la distinción hay que hacerla **a mano dentro del combinador** que le pasás al esquema, no esperar que el esquema la resuelva sola.

3. Reconocer la forma del problema

La mayoría de los ejercicios de "definir función sobre tipo recursivo" son una de estas formas, aunque el nombre del tipo y el dominio cambien de parcial a parcial. Antes de escribir, identificá cuál es:

- **Lista disfrazada** (un caso base "vacío" + un caso recursivo que agrega un elemento y sigue). Las funciones típicas sobre esto son reinventaciones de `length`, `sum`, `maximum`, `(++)` con otro nombre — si reconocés el patrón, no hace falta inventar el algoritmo de cero.
- **Dos ramas que corren en simultáneo** (arrancan del mismo punto, avanzan "a la par"). Se combinan elemento a elemento; cuando una termina antes que la otra, la que sigue continúa sola sin nada que combinar.
- **Dos ramas donde una alimenta a la otra** (secuencialidad: "lo que sale de recorrer la primera es el punto de partida para recorrer la segunda"). Si la primera falla, la segunda ni se intenta — el resultado global es el fallo, sin explorar más.
- **Ramificación codificada con solo dos campos recursivos** (un tipo que conceptualmente tiene "N alternativas posibles" pero el constructor solo trae dos punteros recursivos). Ahí un campo representa "la continuación real" y el otro representa "otra alternativa al mismo nivel" — son roles distintos, no intercambiables. Al recorrer este tipo, lo que se propaga por la "continuación real" no se propaga por la "alternativa", y viceversa.
- **Colapsar/simplificar cadenas de constructores repetidos**. El patrón que funciona es recursar primero al hijo, y después mirar si el resultado ya procesado matchea el mismo constructor para seguir fusionando — así colapsás cadenas de largo arbitrario, no solo pares consecutivos.
- **Simplificar solo en un caso muy puntual y específico**. Si el enunciado describe una condición sintáctica exacta ("cuando la expresión es tal forma particular"), la única manera segura de detectarla es hacer pattern matching de esa forma exacta. **Nunca** evalúes/ejecutes la expresión con un valor "de prueba" inventado para adivinar si estás en ese caso — si la expresión depende de algo que no conocés en ese punto, evaluarla puede no solo fallar, sino darte un resultado **incorrecto y silencioso** (si por casualidad el resultado "parece" válido). Un fallo explícito siempre es preferible a un resultado silenciosamente equivocado.
- **Fusionar dos secuencias ya ordenadas en una sola**. El patrón de "dos punteros": comparás las cabezas actuales de cada secuencia y avanzás **solo** la que ya "ganó" esa comparación — la otra se queda pendiente, porque todavía no se comparó contra lo que sigue. Buscar en lo ya generado (`elem` o similar) nunca hace falta si el problema te garantiza orden y ausencia de repetidos: esa garantía es justamente lo que te permite no buscar.
- **Contar pasos, tamaño o altura sobre una estructura con ramificación**. Definí explícitamente qué cuenta como una unidad (¿el nodo de ramificación en sí suma un paso?) y aplicá cada incremento **solo** al lado de la estructura que efectivamente lo atraviesa. Es un error común "repartir" un mismo incremento a ramas alternativas que en la práctica no pasan por ese punto.

4. Estructura de una demostración

Las 3 columnas

```
double n
=
2 * n

def doble, x <- n
```

Columna 1: el signo `=`. Columna 2: el paso (expresión). Columna 3: la justificación (qué definición/HI/lema se aplicó).

Plantilla completa, en orden, con las frases exactas

Esto es lo que se escribe, en este orden, con estas palabras. Reemplazar lo que está entre `< >`.

Paso 0 — solo si la propiedad tiene un parámetro extra que NO es el tipo recursivo (algo tipo "para todo `k`, ..." o "para todo `env`, ..." al lado de una igualdad de funciones). Fijalo antes de arrancar:

```
Sea <var> :: <tipo> un/a <qué es> cualquiera, fijo y arbitrario (dado por el "para todo <var>" del enunciado).
```

Paso 1 — principio de extensionalidad (cuando la propiedad está escrita como igualdad de funciones, sin nombrar el argumento del tipo recursivo — point-free):

```
Por principio de extensionalidad, siendo <var> un/a <TIPO RECURSIVO> cualquiera, quiero ver que:
{ para todo <var>. <LADO IZQUIERDO> <var> = <LADO DERECHO> <var> ?
```

Paso 2 — desarmar operadores punto-libre si hace falta (compose, flip, etc. — uno por cada operador que quede aplicado):

```
por def. de compose, es equivalente decir:
{ para todo <var>. <forma desarmada izquierda> = <forma desarmada derecha> ?
```

Paso 3 — anunciar qué tipo de demo sigue. Si es sobre la estructura de un tipo de dato:

```
por principio de induccion estructural sobre la estructura de <var>
```

Si es un análisis de casos sobre valores (no sobre un tipo recursivo — por ejemplo, sobre qué constructor concreto puede tener un `Value`, o sobre `True` / `False` de un booleano):

```
por analisis de casos sobre <var1>, <var2>, ...
```

Paso 4 — planteo de los casos (uno por cada constructor del tipo, o uno por cada caso del análisis). Primero se listan TODOS, sin desarrollar:

```
casoBase<N>) siendo <parámetros> un/a <tipo> cualquiera. <var> = <Constructor parámetros>
{ <LADO IZQUIERDO sustituido> = <LADO DERECHO sustituido> ?

casoInductivo<N>) siendo <parámetros> cualquiera. <var> = <Constructor con sub-datos recursivos>
HI1) { <la propiedad, para el primer sub-dato recursivo> !
HI2) { <la propiedad, para el segundo sub-dato recursivo> !
TI) { <LADO IZQUIERDO sustituido> = <LADO DERECHO sustituido> ?
```

(Un solo `HI`) sin numerar si el constructor tiene un solo campo recursivo; `HI1` / `HI2` si tiene dos.)

Si es análisis de casos (no estructural), el planteo de cada caso lleva su propia condición en vez de un constructor:

```
c<N>) siendo <parámetros> cualquiera. <var> = <forma concreta>
{ <LADO IZQUIERDO sustituido> = <LADO DERECHO sustituido> ?
```

Para el caso "todo lo demás" (complemento de los anteriores), escribir la condición como disyunción de negaciones, nunca conjunción — ver más abajo.

Paso 5 — desarrollar cada caso, uno por uno:

```
-- Demuestro:
casoBase<N>)
LI)
  <expresión inicial>
=
  <justificación>
  <expresión siguiente>
LD)
  <expresión inicial>
=
  <justificación>
  <expresión siguiente>
Vale para este caso.
```

Para un `casoInductivo`, la única diferencia es que en algún paso de la cadena la justificación va a decir `HI`, `HI1`, `HI1` y `HI2`, o el nombre de un lema, en vez de `<funcion>.<n>`.

Paso 6 — cierre final, después de que todos los casos dieron bien:

La propiedad vale.

Frases sueltas, por situación (tabla rápida)

Situación	Frase exacta a escribir
Fijar el parámetro extra antes de extensionalidad	Sea <code><var> :: <tipo></code> cualquiera, fijo y arbitrario (dado por el "para todo <code><var></code> " del enunciado).
Aplicar extensionalidad	Por principio de extensionalidad, siendo <code><var></code> un/a <code><TIPO></code> cualquiera, quiero ver que:
Desarmar <code>(.)</code>	por def. de compose, es equivalente decir:
Desarmar <code>flip</code>	por def. de flip, es equivalente decir: (o como justificación de un solo paso: <code>flip</code>)
Anunciar inducción estructural	por principio de induccion estructural sobre la estructura de <code><var></code>
Anunciar análisis de casos	por analisis de casos sobre <code><var1></code> , <code><var2></code> , ...
Nombrar un caso base	<code>casoBase<N>)</code> siendo <code><parámetros></code> cualquiera. <code><var> = <Constructor></code>
Nombrar un caso inductivo	<code>casoInductivo<N>)</code> siendo <code><parámetros></code> cualquiera. <code><var> = <Constructor></code>
Hipótesis inductiva	<code>HI<n>)</code> ; <code><propiedad> !</code>
Tesis inductiva	<code>TI)</code> ; <code><propiedad> ?</code>
Objetivo de un caso base	<code>{ <propiedad> ?</code> (sin HI, va directo el objetivo)
Justificar un paso por definición	<code><funcion>.<n></code> (número de la ecuación de esa función que usaste, contando desde la primera)
Justificar un paso por HI	<code>HI / HI1 / HI1 y HI2</code>
Justificar un paso por un lema	<code><nombreDelLema></code>
Cerrar un caso que cierra bien	Vale para este caso. (equivalente: Para este caso vale.)

Cerrar toda la demostración	La propiedad vale.
Nombrar un sub-caso en análisis de casos	$c(N)$ siendo $\langle \text{parámetros} \rangle$ cualquiera. $\langle \text{var} \rangle = \langle \text{forma concreta} \rangle$
Plantear el caso "todo lo demás"	$c(N)$ $\langle \text{condición 1} \rangle$ ó $\langle \text{condición 2} \rangle$ — nunca "y" (ver abajo por qué)

Extensionalidad: dos cuantificadores, no uno

Cuando la propiedad viene dada como igualdad de funciones point-free con algún parámetro extra de por medio (Paso 0 de la plantilla), hay **dos** variables en juego:

- La explícita (el parámetro extra): se queda **fija y arbitraria** durante toda la demo. No es sobre ella que inducís.
- La implícita, revelada por principio de extensionalidad: " $f = g$ " significa "para todo x , $f\ x = g\ x$ " — y es sobre **ese x** que hacés inducción estructural. Los `casoBase` / `casoInductivo` son casos del tipo recursivo, no del parámetro extra que quedó fijo.

Antes de arrancar a hacer casos, identificá cuál de las dos variables de la propiedad es la que tiene el tipo recursivo (esa es sobre la que inducís) y cuál es la que se queda fija.

Usar la HI (o un lema) correctamente

- **Sustituís toda la expresión**, no dejás nada de más alrededor. Si la HI dice $f\ x = g\ x$, reemplazar una ocurrencia de $f\ x$ te da directamente $g\ x$ — no un wrapper extra alrededor de $g\ x$.
- **Señal de alarma:** si un `casoInductivo` "cierra" sin haber usado la HI en ningún paso, algo está mal — probablemente falta abrir un caso, o se está asumiendo algo que en realidad depende de ella.
- **Lemas auxiliares** sirven para sacar de la demo principal una sub-demostración que se repetiría en varios casos. Pero el lema **necesita su propia demo** — no es gratis solo por enunciarlo. Si la función de la que depende el lema no es recursiva, el lema tampoco necesita inducción propia: alcanza con abrir los casos que correspondan a las ecuaciones de esa función.

Demo por casos (no estructural)

- Los casos tienen que ser **conjuntamente exhaustivos**. Si un caso es "se cumple A y se cumple B", el complemento correcto es "no se cumple A **o** no se cumple B" (al menos una falla) — no "y" (las dos fallan), que deja afuera los casos mixtos.
- **No fijes un valor concreto cuando el caso pide "para todo x tal que..."**. Si tenés que probar algo para "cualquier par de valores que cumplan cierta condición negativa", no elijas un ejemplo particular — eso prueba una instancia, no el caso general. Si la función que estás usando tiene una cláusula comodín que no distingue entre formas, no necesitás abrir más sub-casos: la hipótesis del caso ya es exactamente la condición que dispara ese comodín, para cualquier forma concreta que tengan los valores.

Demo con `if` (caso general: casarte sobre la condición)

Si una función está definida como $f\ x = \text{if } \langle \text{cond} \rangle \text{ then } \langle e1 \rangle \text{ else } \langle e2 \rangle$ y necesitás desarrollar $f\ x$ dentro de una demo, la única forma de "abrir" el `if` es casarte sobre el valor de verdad de $\langle \text{cond} \rangle$. No hay atajo por default — el atajo (más abajo) solo existe en una situación particular.

por análisis de casos sobre <cond>

caso1) <cond> = True

LI)

f x

= def. f, ya que <cond> = True (entra por la rama del then)

<e1>

= <siguiente justificación>

...

caso2) <cond> = False

LI)

f x

= def. f, ya que <cond> = False (entra por la rama del else)

<e2>

= <siguiente justificación>

...

Una vez que fijaste el valor de verdad, el `if` deja de ser un `if` — se convierte directamente en la rama que corresponde (`<e1>` o `<e2>`), y desde ahí seguís la cadena de igualdades como con cualquier otra expresión. Es exhaustivo por construcción: un booleano es `True` o `False`, no hay tercera opción, así que acá no hace falta preocuparse por AND/OR como en un análisis de casos sobre un tipo con más de dos formas ("Demo por casos", más arriba).

Atajo — solo cuando tenés un `if` anidado adentro de otro `if`, y el `else` de ambos niveles es exactamente la misma expresión repetida. Ahí, en vez de abrir los 4 casos posibles (`b1`, `b2` en `True/True`, `True/False`, `False/True`, `False/False`), podés citar directamente:

```
if b1 then (if b2 then e1 else e2) else e2 = if (b1 && b2) then e1 else e2
```

Si el `else` de los dos niveles **no** es la misma expresión, el atajo no aplica — hay que volver al caso general de arriba y abrir los casos de verdad (en este caso, potencialmente los 4 combinados de `b1 / b2`) uno por uno.

5. Funciones de Prelude que aparecen todo el tiempo

Función	Tipo	Para qué sirve
<code>map</code>	<code>(a -> b) -> [a] -> [b]</code>	aplicar una función a cada elemento, preservando la forma de la lista
<code>filter</code>	<code>(a -> Bool) -> [a] -> [a]</code>	quedarte con los elementos que cumplen un predicado
<code>flip</code>	<code>(a -> b -> c) -> b -> a -> c</code>	invertir el orden de los primeros dos argumentos de una función — típico cuando tenés <code>f :: A -> B -> C</code> pero necesitás pasarla como <code>B -> C</code> con <code>A</code> ya fijo
<code>(.)</code> (compose)	<code>(b -> c) -> (a -> b) -> a -> c</code>	encadenar dos funciones sin nombrar el argumento intermedio; <code>(f . g) x = f (g x)</code>
<code>id</code>	<code>a -> a</code>	no hacer nada — útil como placeholder cuando necesitás pasar una función pero no debería importar cuál (con cuidado: no siempre es seguro usarlo para "simular" un valor real, ver §3)
<code>const</code>	<code>a -> b -> a</code>	ignora el segundo argumento, devuelve siempre el primero — típico para armar un valor constante independiente del input
<code>flip const</code>	<code>a -> b -> b</code>	el opuesto de <code>const</code> : ignora el primero, devuelve siempre el segundo. No tiene nombre propio en el Prelude — se escribe <code>flip const</code> o directo <code>_ y -> y</code>
<code>subtract</code>	<code>Num a => a -> a -> a</code>	<code>subtract n = \y -> y - n</code> (resta <code>n</code> , en ese orden). Existe porque <code>(- n)</code> como sección no funciona — Haskell lo lee como el número negativo <code>-n</code> , no como una función. Si necesitás "restar <code>n</code> " en punto libre (ej. para pasarle a <code>map</code>), <code>subtract n</code> es la forma correcta, <code>(- n)</code> no
<code>curry</code>	<code>((a,b) -> c) -> a -> b -> c</code>	convierte una función que toma un par en una función currificada (dos argumentos separados en vez de uno solo tipo tupla)
<code>uncurry</code>	<code>(a -> b -> c) -> (a,b) -> c</code>	la inversa de <code>curry</code> : convierte una función currificada en una que toma un solo par
<code>uflip</code>	<code>(a -> b -> c) -> (a,b) -> c</code>	"flip sin curricular": toma los dos argumentos como un par y se los pasa invertidos a <code>f</code> — <code>uflip f (x,y) = f y x</code> . No es un nombre estándar de Haskell (es <code>uncurry</code> compuesto con <code>flip</code> , <code>uncurry (flip f)</code>), así que fijate si tu cátedra lo define distinto
<code>apply</code> <code>((\$))</code>	<code>(a -> b) -> a -> b</code>	aplicar una función a un argumento escrito como función en vez de yuxtaposición — <code>apply f x = f x</code> . En Haskell esto ya existe como el operador <code>(\$)</code> , útil sobre todo para no encadenar paréntesis: <code>f \$ g \$ h x</code> en vez de <code>f (g (h x))</code>
<code>twice</code>	<code>(a -> a) -> a -> a</code>	aplicar la misma función dos veces seguidas — <code>twice f = f . f</code> , o punto a punto <code>twice f x = f (f x)</code>
<code>replicate</code>	<code>Int -> a -> [a]</code>	lista de <code>n</code> copias del mismo elemento — cuidado, tiene que ser una lista de <code>n</code> elementos, no un único elemento que contenga <code>n</code> cosas adentro
<code>null</code>	<code>[a] -> Bool</code>	¿la lista está vacía? — útil para distinguir a mano los dos sentidos de un caso base compartido en un <code>fold</code> (ver §2)
<code>(++)</code>	<code>[a] -> [a] -> [a]</code>	concatenar listas — suele ser lo que hace por debajo una secuencia/concatenación de dos sub-resultados
<code>zip</code>	<code>[a] -> [b] -> [(a,b)]</code>	empareja dos listas elemento a elemento en tuplas — se corta en la lista más corta, no acarrea el resto de la más larga
<code>zipWith</code>	<code>(a -> b -> c) -> [a] -> [b] -> [c]</code>	como <code>zip</code> , pero combinando cada par con una función en vez de armar la tupla — <code>zipWith f xs ys = map (uncurry f) (zip xs ys)</code> . Misma salvedad: se corta en la más corta, no es lo mismo que "acarrear el resto" (ver "dos ramas que corren en simultáneo" en §3)
<code>maybe</code>	<code>b -> (a -> b) -> Maybe a -> b</code>	destructor de <code>Maybe</code> : <code>maybe defecto f Nothing = defecto</code> , <code>maybe defecto f (Just x) = f x</code> — evita escribir el <code>case</code> a mano

<code>either</code>	<code>(a -> c) -> (b -> c) -> Either a b -> c</code>	lo mismo que <code>maybe</code> pero para <code>Either</code> — aplica una función u otra según qué lado tenga el valor
---------------------	---	---

6. Sintaxis que conviene tener fresca

`let ... in`

```
let x = expr1
    y = expr2
in expr3
```

- Es una **expresión** — va en cualquier lugar donde puede ir un valor (adentro de un `if`, como argumento de una función, etc.).
- Todo `let` necesita su `in`. No se pueden encadenar dos `let` sin cerrar el primero:

```
-- MAL: al primer let le falta el in
let res = f x y
let final = res + 1
in final

-- BIEN: un solo let con dos bindings alineadas
let res    = f x y
    final = res + 1
in final
```

- Varias bindings en el mismo `let` van alineadas en columna (mismo margen izquierdo), sin `,` ni `;` entre ellas — el layout de Haskell las separa solo por la indentación.

`where`

```
f x
| cond1 = a
| cond2 = b
where
  a = ...
  b = ...
```

- **No** es una expresión — se pega después de una ecuación completa (una definición de función o un patrón), no se puede meter suelto en el medio de otra expresión como el `let`.
- El scope de un `where` cubre **todas** las guards y el cuerpo de esa ecuación — por eso conviene cuando el mismo valor auxiliar hace falta en más de una guard.
- Si la función tiene varias ecuaciones (varios patrones distintos), cada ecuación tiene su propio `where` — no se comparte entre ecuaciones.

`case ... of`

```
case expr of
  Patron1  -> resultado1
  Patron2 x -> resultado2  -- podés bindear variables del patrón acá
  _        -> resultadoDefault
```

- Es una **expresión** (da un valor) — a diferencia del pattern matching a nivel de ecuaciones de función, se puede usar sobre cualquier valor intermedio sin tener que nombrarlo como argumento de una función nueva.
- Los patrones se prueban en orden, de arriba hacia abajo — gana el primero que matchea.

Guards (|)

```
f x
| cond1 = resultado1
| cond2 = resultado2
| cond3 = resultado3
```

- Se evalúan en orden, de arriba hacia abajo — gana la primera condición que dé `True`.
- Si ninguna guard matchea y no hay un catch-all, es error en tiempo de ejecución. En vez de agregar `otherwise` como última guard, una alternativa es cerrar con una ecuación aparte de patrón comodín (`f _ = ...`) después de las guards, o un patrón `_` si estás dentro de un `case`.

Lambda (`\x -> ...`)

```
\x -> x + 1
\x y -> x && y
```

- Función anónima, sin nombre — útil para pasarle a otra función un comportamiento "de una sola vez" (a `map`, `filter`, un campo de tipo función de un constructor, etc.) sin definirlo aparte.
- Puede tomar más de un argumento: `\x y -> ...` es lo mismo que `\x -> \y -> ...`.

Secciones

```
(+1)  -- \x -> x + 1
(1+)  -- \x -> 1 + x
(2>)  -- \x -> 2 > x
(>2)  -- \x -> x > 2
```

- Aplicación parcial de un operador infijo, escribiendo solo uno de los dos lados entre paréntesis.
- Ojo con `-`: `(- n)` **no** es una sección, Haskell lo lee como el número negativo `-n`. Para "restar `n`" como función usá `subtract n` (ver §5).

Apéndice — plantilla de demo en .hs

```
{-
a.  para todo xs. para todo ys.      length (xs ++ ys) = length xs + length ys

Prop.: pregunta del enunciado
Demo.: por ppio de induccion estructural sobre la estructura a demostrar, es equivalente demostrar
que

Sea uno o varios nombres de variables distintas a la pregunta :: tipo de la var

CB. asignacion de caso base en las vars de arriba
    reemplazas las vars...
    ¿length ([] ++ []) = length [] + length []?

CI. asignacion de la estructura abierta en las vars de arriba
    HI) cambias los signos de pregunta por signos de exclamacion de la pregunta inicial
    reemplazas las vars...
    TI) ¿length ((z: zs) ++ (a: as)) = length (z: zs) + length (a: as)?

Dem. caso base)
    Lado izq:
        ...
    Lado der:
        ...

    Vale o no vale

Dem. caso inductivo)
    Lado izq:
        ...
    Lado der:
        ...

    Vale o no vale

-}
```